

# Algoritmos de Pesquisa e Ordenação Interna

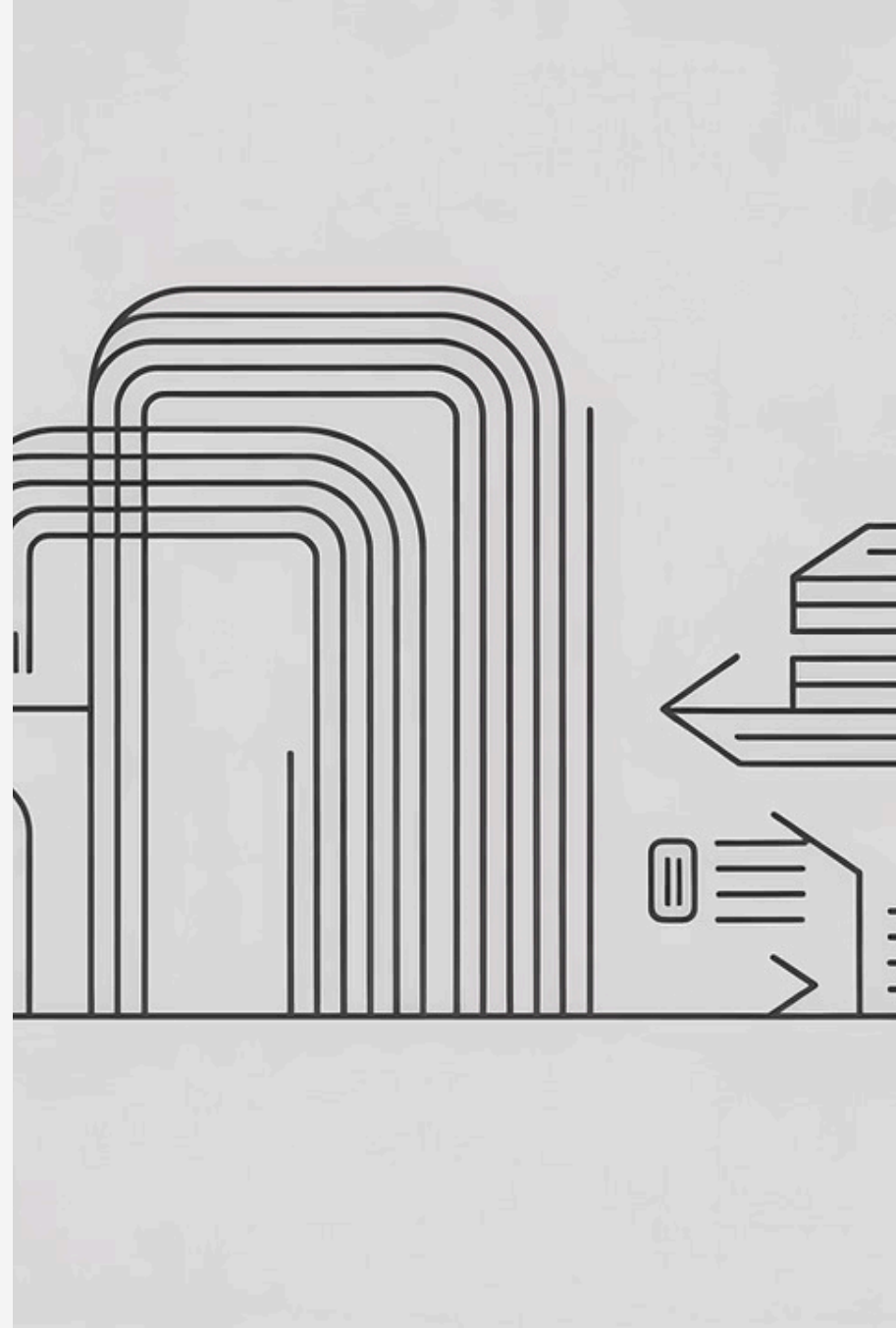
## Universidade Federal de Goiás

Instituto de Matemática e Estatística e Instituto de Informática (INF/UFG)

Curso: Matemática Aplicada e Computacional

Disciplina: INF0286 – Algoritmos e Estruturas de Dados I

Professora: Dra. Renata Dutra Braga



# Objetivos da Aula

O objetivo é que, ao final dessa aula, o aluno seja capaz de:

## Explicar

como funcionam os algoritmos de busca sequencial e busca binária.

## Implementar

em linguagem C, cada um desses algoritmos usando funções.

## Entender

o que é ordenação interna, ou seja, ordenação realizada diretamente na memória principal.

## Implementar o Bubble Sort

que é um dos algoritmos de ordenação interna mais didáticos.

## Analisar a complexidade assintótica

reconhecendo quando um algoritmo é  $O(1)$ ,  $O(n)$ ,  $O(\log n)$  ou  $O(n^2)$ .

Esses conceitos são essenciais para avaliar desempenho e escolher o algoritmo adequado.

# Motivação

Pesquisar e ordenar dados são operações que aparecem em qualquer sistema, do mais simples ao mais complexo. Um algoritmo mal escolhido pode tornar um sistema lento, principalmente quando o conjunto de dados cresce.



## Vetor não ordenado

Quando o vetor **não está ordenado**, a única forma garantida de busca é a **sequencial**, percorrendo item por item.



## Ordenação interna

E para ter um vetor ordenado, precisamos aplicar algum algoritmo de **ordenação interna**, que ocorre quando todos os dados cabem na memória RAM.

Por isso, é fundamental estudar esses três tipos de algoritmos em conjunto.



## Vetor ordenado

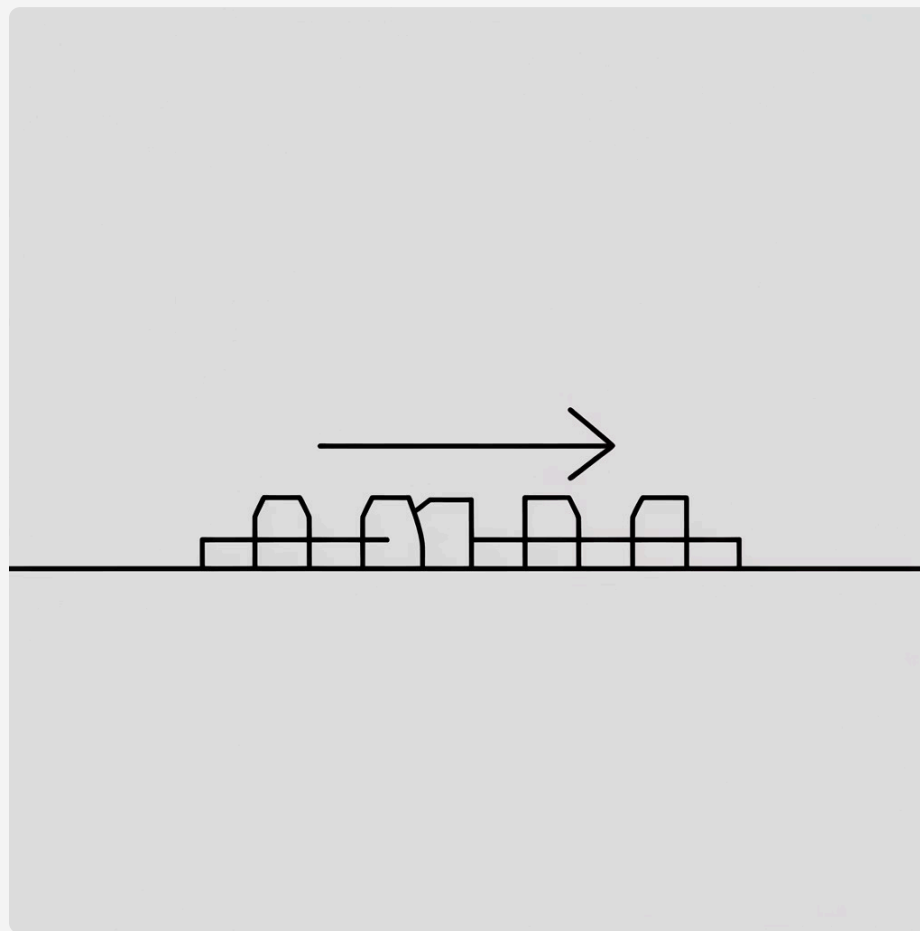
Quando o vetor **está ordenado**, podemos usar a **busca binária**, que é muito mais rápida.



# Busca Sequencial: Conceito

A **busca sequencial** é o método mais direto de procurar um elemento em um vetor.

- Ela **não exige ordenação**, o que é uma vantagem.
- Percorre o vetor **elemento por elemento**, da primeira até a última posição.
- É simples de implementar e entender.
- Porém, quando o vetor é grande, torna-se **ineficiente**, porque pode ser necessário percorrer todas as posições.



# Busca Sequencial: Código em C

A busca sequencial funciona da seguinte maneira: percorre o vetor desde o início, comparando cada elemento com a chave que estamos procurando. Se houver uma coincidência, retorna a posição. Se chegar ao final sem encontrar, retorna -1.

## Código em C:

```
int buscaSequencial(int v[], int n, int chave) {  
    for (int i = 0; i < n; i++) {  
        if (v[i] == chave) {  
            return i;  
        }  
    }  
    return -1;  
}
```

## Complexidade:

- Melhor caso:  $O(1)$ , se encontrar na primeira posição.
- Caso médio e pior caso:  $O(n)$ , porque pode ser necessário percorrer todo o vetor.

Essa busca é indicada para vetores pequenos ou vetores que ainda não estão ordenados.

# Busca Sequencial: Complexidade

Aqui reforçamos os valores:

**$O(1)$**

**Melhor caso**

**$O(n)$**

**Caso médio**

**$O(n)$**

**Pior caso**

**$O(1)$**

**Espaço**

porque usa apenas variáveis  
simples e nenhuma estrutura  
adicional.

# Busca Binária: Conceito

A **busca binária** só pode ser utilizada se o vetor estiver **ordenado**.

O funcionamento é simples:

01

---

**Encontramos o meio do vetor.**

02

---

**Comparamos o valor buscado com o valor do meio.**

03

---

**Se for igual, terminamos.**

04

---

**Se for menor, continuamos buscando na metade esquerda.**

05

---

**Se for maior, buscamos na metade direita.**

A cada passo, o espaço de busca é reduzido pela metade, tornando o processo muito mais rápido que a busca sequencial.

# Busca Binária: Código em C

```
int buscaBinaria(int v[], int n, int chave) {  
    int inicio = 0, fim = n - 1;  
    while (inicio <= fim) {  
        int meio = (inicio + fim) / 2;  
        if (v[meio] == chave) return meio;  
        else if (chave < v[meio]) fim = meio - 1;  
        else inicio = meio + 1;  
    }  
    return -1;  
}
```

Aqui percebemos que a cada iteração reduzimos pela metade os elementos que ainda podem conter a chave. Isso é possível somente porque o vetor está ordenado.



# Busca Binária: Complexidade

## **Melhor caso: $O(1)$**

se o elemento estiver exatamente no meio do vetor.

## **Caso médio e pior caso: $O(\log n)$**

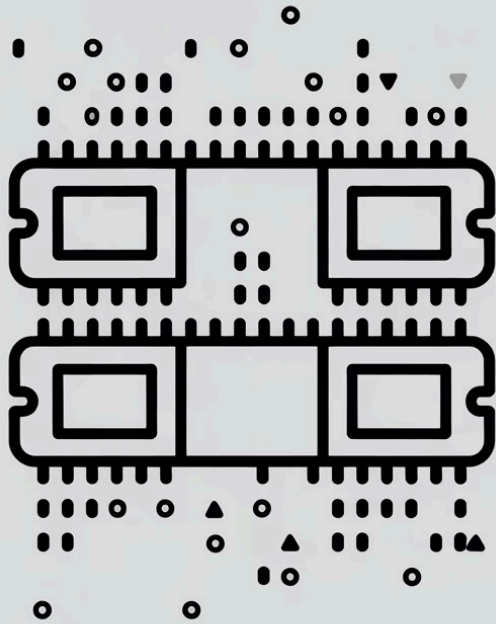
pois o vetor é dividido repetidamente por dois.

## **Espaço: $O(1)$**

assim como na busca sequencial.

Essa eficiência faz a busca binária ser muito superior quando os dados já estão ordenados.

# Ordenação Interna: Conceito



Ordens internas são algoritmos de ordenação executados totalmente na memória principal. Seu objetivo é colocar os elementos em uma determinada ordem — normalmente crescente — para facilitar operações posteriores, como busca binária.

O **Bubble Sort** é um dos algoritmos mais didáticos, pela sua simplicidade, apesar de não ser eficiente para grandes vetores.

# Bubble Sort: Código em C

O Bubble Sort funciona comparando **pares adjacentes** e trocando-os quando estão fora de ordem. Assim, os maiores elementos "sobem" para o final.

## Código:

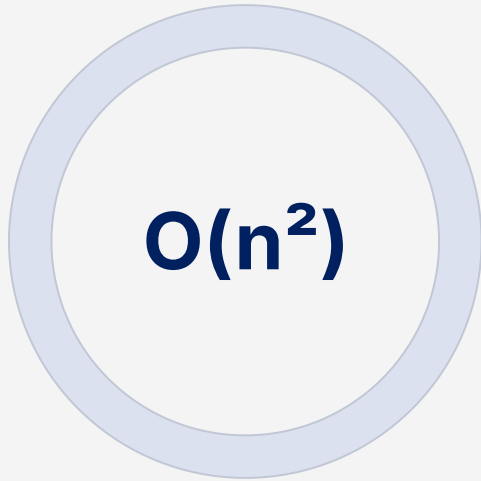
```
void bubbleSort(int v[], int n) {  
    for (int i = 0; i < n - 1; i++) {  
        for (int j = 0; j < n - 1 - i; j++) {  
            if (v[j] > v[j+1]) {  
                int temp = v[j];  
                v[j] = v[j+1];  
                v[j+1] = temp;  
            }  
        }  
    }  
}
```

## Complexidade:

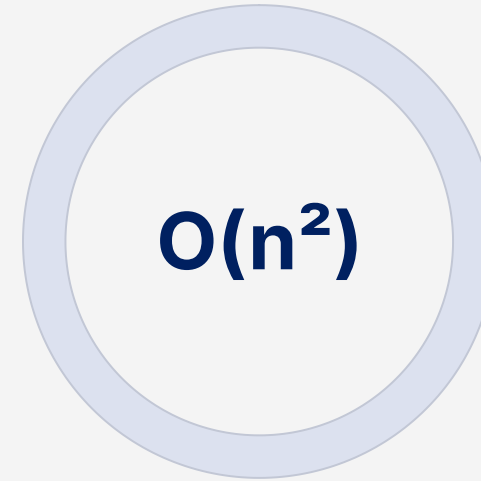
- Caso médio e pior caso:  $O(n^2)$
- Melhor caso na versão simples:  $O(n^2)$  também

Mesmo sendo ineficiente, é excelente para fins didáticos e ajuda a compreender a lógica de ordenação.

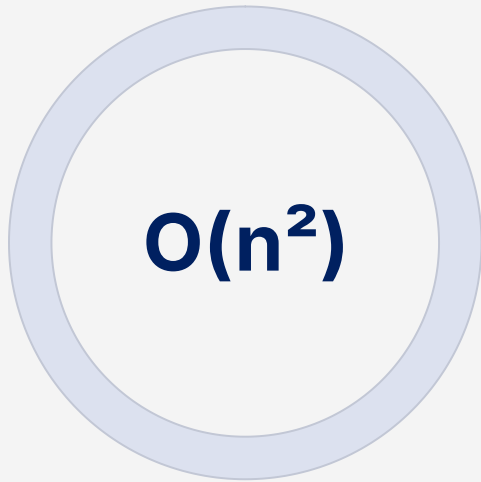
# Bubble Sort: Complexidade



**Melhor caso**



**Caso médio**



**Pior caso**



**Espaço**

pois a ordenação é feita diretamente no vetor.

# Comparação dos Algoritmos

A tabela comparativa resume os algoritmos estudados:

| Algoritmo        | Complexidade | Requisito      |
|------------------|--------------|----------------|
| Busca Sequencial | $O(n)$       | Nenhum         |
| Busca Binária    | $O(\log n)$  | Vetor ordenado |
| Bubble Sort      | $O(n^2)$     | Nenhum         |

☐ E reforça a observação fundamental: A busca binária só pode ser aplicada se o vetor estiver ordenado, o que muitas vezes exige uma fase de ordenação prévia.

# Programa Completo de Demonstração

Este slide mostra um exemplo completo de uso de todos os algoritmos juntos.

```
int main() {  
    int vetor[] = {42, 7, 15, 3, 9, 81, 14};  
    int n = 7, chave = 15;  
  
    int pos = buscaSequencial(vetor, n, chave);  
  
    bubbleSort(vetor, n);  
  
    pos = buscaBinaria(vetor, n, chave);  
}
```

O programa demonstra o fluxo natural:



**Vetor original.**



**Busca sequencial.**



**Ordenação com Bubble Sort.**



**Busca binária no vetor agora ordenado.**

Mostra também a importância da ordenação interna para que possamos usar algoritmos de busca mais eficientes.

# Conclusão

**Dominar algoritmos de pesquisa e ordenação é fundamental para o desenvolvimento de sistemas eficientes**

## **Busca Sequencial**

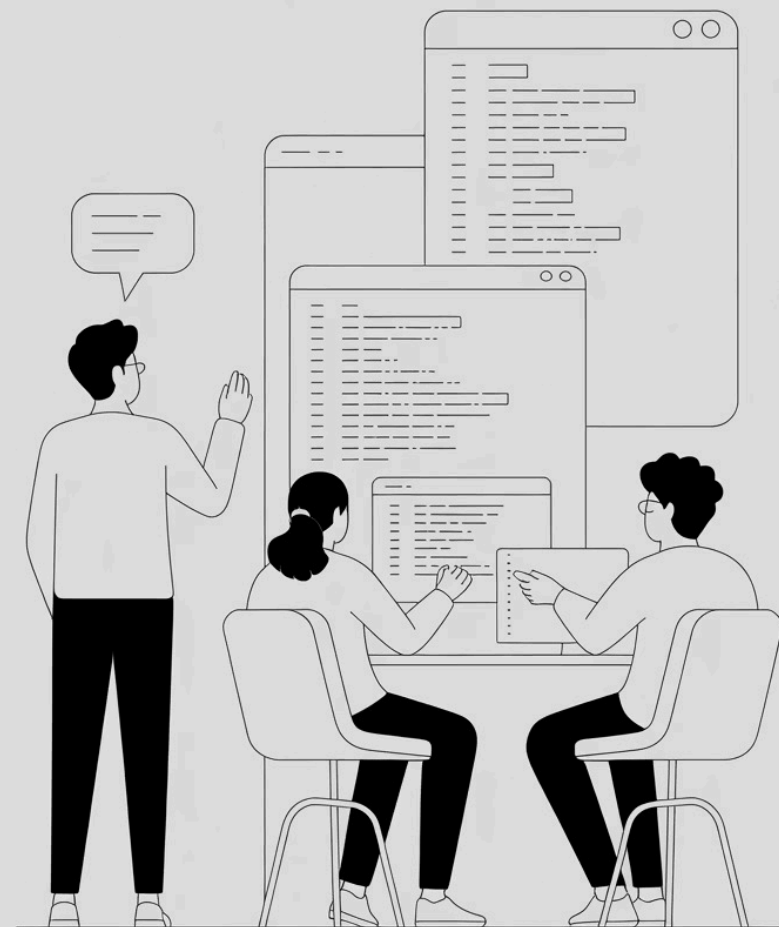
Simple e universal,  
mas limitada em  
performance

## **Busca Binária**

Extremamente  
eficiente, mas requer  
ordenação prévia

## **Bubble Sort**

Didático e intuitivo, ideal  
para aprendizado



# Vamos praticar?

Exercício em C?



**Revisar  
Sintaxe  
Básica**



**Praticar  
Estruturas**



**Resolver  
Algoritmos**



**Depurar  
Código**